This will redirect the user to the main action if they tried to access a legacy action. The intention was to preserve the URL parameters to the legacy action and pass them to the main action. However, it can be exploited by attacker if they included a host key in the URL:

```
http://www.example.com/site/legacy?param1=xy&param2=23&host=www.attacker.com
```

If it is at the end of the URL it will hardly be noticed and redirects the user to the attacker.com host. A simple countermeasure would be to *include only the expected parameters in a legacy action* (again a whitelist approach, as opposed to removing unexpected parameters). *And if you redirect to an URL, check it with a whitelist or a regular expression*.

### 4.1.1 Self-contained XSS

Another redirection and self-contained XSS attack works in Firefox and Opera by the use of the data protocol. This protocol displays its contents directly in the browser and can be anything from HTML or JavaScript to entire images:

```
data:text/html;base64,PHNjcmlwdD5hbGVydCgnWFNTJyk8L3NjcmlwdD4K
```

This example is a Base64 encoded JavaScript which displays a simple message box. In a redirection URL, an attacker could redirect to this URL with the malicious code in it. As a countermeasure, *do not allow the user to supply (parts of) the URL to be redirected to*.

## 4.2 File Uploads

*Make sure file uploads don't overwrite important files, and process media files asynchronously.*

Many web applications allow users to upload files. *File names, which the user may choose (partly), should always be filtered* as an attacker could use a malicious file name to overwrite any file on the server. If you store file uploads at /var/www/uploads, and the user enters a file name like "../../../etc/passwd", it may overwrite an important file. Of course, the Ruby interpreter would need the appropriate permissions to do so - one more reason to run web servers, database servers and other programs as a less privileged Unix user.

When filtering user input file names, *don't try to remove malicious parts*. Think of a situation where the web application removes all "../" in a file name and an attacker uses a string such as "..../" - the result will be "../". It is best to use a whitelist approach, which *checks for the validity of a file name with a set of accepted characters*. This is opposed to a blacklist approach which attempts to remove not allowed characters. In case it isn't a valid file name, reject it (or replace not accepted characters), but don't remove them. Here is the file name sanitizer from the attachment_fu plugin:

```
def sanitize_filename(filename)
  filename.strip.tap do |name|
    # NOTE: File.basename doesn't work right with Windows paths on
Unix
    # get only the filename, not the whole path
    name.sub! /\A.*(\\|\/)/, ''
    # Finally, replace all non alphanumeric, underscore
    # or periods with underscore
    name.gsub! /[^\w\.\-]/, '_'
  end
end
```

A significant disadvantage of synchronous processing of file uploads (as the attachment_fu plugin may do with images), is its *vulnerability to denial-of-service attacks*. An attacker can synchronously start image file uploads from many computers which increases the server load and may eventually crash or stall the server.